SpendSense AI — Procurement Spend Analytics Assistant

GenAI Capstone Project Report (Build-Your-Own-Project)

Student	Muhil Kennedy
Project option	Option 1 — Build Your Own Project (BYOP)
Domain	Supply Chain / Procurement (SAP Ariba context)

---

1. Executive summary

SpendSense AI is a conversational analytics assistant for procurement spend data. A user asks a plain-English question — *"Who were our top 5 suppliers in FY2024?"*, *"How much maverick spend do we have and where?"* — and the assistant uses OpenAI's **function calling** to translate that question into a precise call on a procurement analytics function, executes it over the spend dataset with **pandas**, and replies in natural language with tables and figures.

The core design principle is **grounding**: the language model never invents numbers. It only *chooses* which analytics function to run and with what arguments; the actual computation is done by deterministic Python. This makes every figure traceable and correct, which is essential for a business-reporting tool.

The project ships two front-ends over one shared engine: a **Jupyter notebook** (for walkthrough/teaching) and a **SAP Fiori-styled web app** (for an interactive demo), plus a safety **moderation** layer and a documented, reproducible synthetic dataset.

---

2. Objectives

1. Build a chatbot that answers real procurement questions **accurately** (no hallucinated numbers).
2. Demonstrate **OpenAI function calling** — the key capability highlighted in the course material.

3. Provide a clean, professional **user experience** suitable for a business audience.
4. Add responsible-AI basics: **content moderation** of user input.
5. Document the system thoroughly and make it **reproducible**.

---

3. Problem & motivation

In procurement (e.g. SAP Ariba environments), analysts constantly ask questions of spend data: *which suppliers dominate spend, how much is off-contract ("maverick") spend, which categories are growing, are suppliers delivering on time.* Answering these normally means writing SQL or building BI dashboards. SpendSense lets a non-technical stakeholder get the same answers by **just asking**, while guaranteeing the numbers come from real aggregation rather than a model's guess.

Two procurement-specific metrics are built in: - **Maverick spend** — purchase orders with no contract reference (off-contract buying), a key compliance/savings KPI. - **Supplier on-time performance** — promised vs actual delivery days.

---

4. Data

Source: a **synthetic** dataset generated by `make_data.py` (fixed random seed → fully reproducible). Synthetic data was chosen deliberately: it lets the schema match the analytics functions exactly, carries no licensing/privacy constraints, and is reproducible for the grader. (The course explicitly permits self-collected data.) The schema mirrors a real procurement PO export, so a real dataset can be swapped in by renaming columns — see `README.md`.

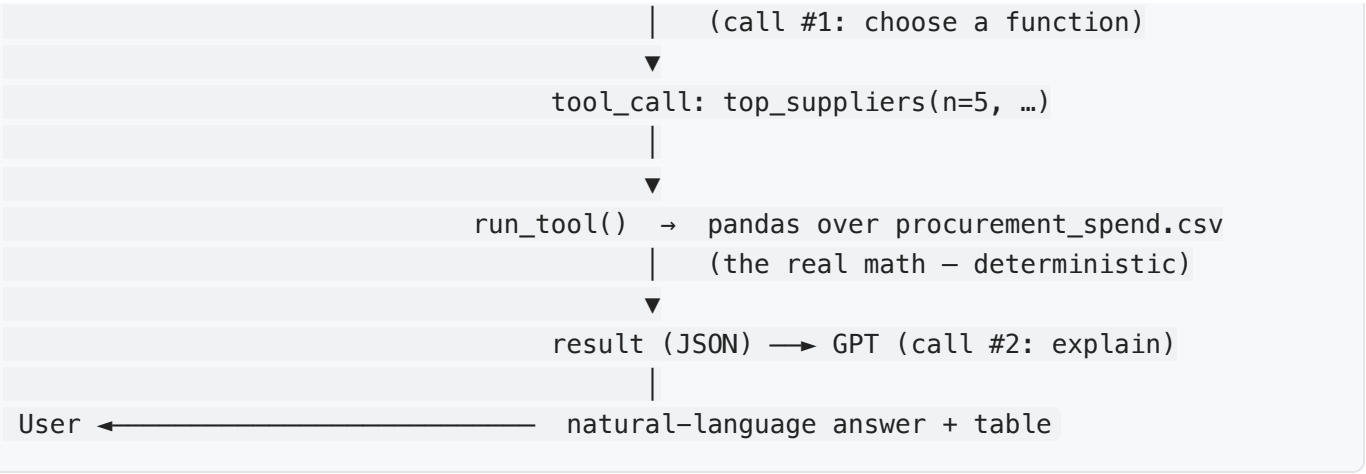
Scale: 2,600 purchase orders · **~$417M** total spend · **8** categories · **37** suppliers · 2 fiscal years (2023-07 to 2025-06). **~28%** of spend is off-contract (maverick).

Key columns: `po_date`, `supplier`, `category`, `department`, `region`, `quantity`, `unit_price`, `amount`, `contract_id` (blank = maverick), `delivery_days_promised`, `delivery_days_actual`, `on_time`.

---

5. System design & architecture

```
          plain-English question
User ──────────────────────────────►  GPT (gpt-4o-mini) + tool schemas
```



Components - **Analytics engine** (`spend_functions.py`): 6 pure pandas functions + their JSON **tool schemas** + a `run_tool()` dispatcher. All returns are JSON-serializable. - **Chat loop** (`spend_chat.py`): moderation gate → function-calling round-trip (loops so GPT may call several tools) → final answer. Also reports which tools were called. - **Front-ends:** the notebook and the Flask web app both import the same engine. - **Safety:** OpenAI moderation on every user input.

A full visual version is in **`ARCHITECTURE.html`**.

The 6 tools

Tool	Answers
`get_total_spend`	total spend with optional date/category/supplier/department/region filters
`top_suppliers`	top-N suppliers by spend
`spend_by_category`	spend split by category (with % of total)
`maverick_spend`	off-contract spend, optionally grouped
`supplier_performance`	on-time rate + average delay for a supplier
`spend_trend`	spend over time (month/quarter)

Why this is a strong design: unlike a rigid staged chatbot, GPT **dynamically** selects the right function and extracts typed arguments itself — no brittle text parsing — and the numbers are always computed by code, never by the model.

---

6. Technical implementation

- **Model:** `gpt-4o-mini` (chat completions).

- **Function calling:** each analytics function is exposed as an OpenAI `tools` entry with a typed JSON-Schema (`enum`s for categories/regions/departments, `minimum` on counts). The loop handles **multi-tool** questions (e.g. maverick % needs both `maverick_spend` and `get_total_spend`) and feeds tool results back for a grounded answer.
- **Dispatcher:** `run_tool(name, args)` maps a tool-call to its Python function and never raises — errors are returned as data so the model can recover.
- **Prompt engineering (4 techniques):** 1. **Role / persona** — "senior procurement spend-analyst". 2. **Grounding / guardrails** — "ALWAYS use a tool; NEVER invent numbers"; explicit data span & fiscal-year definition. 3. **Few-shot** — five *question → tool-call* exemplars. 4. **Chain-of-thought** — a silent 4-step reasoning recipe (metric → filters → tool → summarize).
- **Moderation:** `openai.moderations` screens each input; flagged messages are blocked with a safe refusal (fails *open* on API error so the app stays usable).
- **Web app:** Flask + AJAX (`/chat` returns JSON; no full-page reloads), SAP Fiori styling, suggestion chips, a typing indicator, and **tool-transparency chips** showing which function was called. A small offline Markdown renderer formats tables without any external JS.
- **Efficiency:** the dataset is loaded once and cached; all aggregation is vectorized pandas; the model runs at `temperature=0` for deterministic, reproducible answers.

---

7. User experience

- Natural-language Q&A; answers use **bold figures** and **Markdown tables**.
- **Suggestion chips** give first-time users example questions in one click.
- **Tool chips** under each answer build trust ("🔧 top_suppliers(n=5)").
- Graceful handling of empty/filtered-to-nothing results and of unsafe input.
- Multi-turn context (follow-up questions work).

---

8. Quality assurance

- **Local smoke tests** of all six functions on the real CSV (ranges, filters, dispatch, JSON output).
- **Live tests** of the full function-calling loop against the OpenAI API.
- **Adversarial multi-agent code review:** six independent reviewers each audited the analytics engine through a different lens (dates, aggregation, maverick/on-time, schema-consistency, edge-cases, domain). Five found nothing; one found a real edge case (below), which was fixed and re-verified.

---

9. Challenges & how they were solved

Challenge	Resolution
Negative `n` edge case — `top_suppliers(n=-2)` used `df.head(-2)`, which pandas reads as "all but last 2" (35 rows).	Clamp with `head(max(0, n))` **and** add `"minimum": 1` to the tool schema. Verified.
Model guessed a date range when the user gave none → under-counted.	Prompt rule: "if no period is named, call tools **without** date filters (use all data)."
Inconsistent tool output shape (early testing) — model sometimes wrapped results.	Deterministic Python functions return a fixed JSON shape; the model only *reads* them.
Deprecated OpenAI pieces (from the course era): `gpt-3.5-turbo` retired, old SDK removed.	Used current SDK + `gpt-4o-mini`; verified module-level calls still work.
Grounding / hallucination risk for a reporting tool.	Hard guardrail + function calling means every number is computed, not generated.

---

10. Lessons learned

- **Function calling shifts the model's job** from "answer" to "route": it decides *what* to compute, and deterministic code does the computing. This is the right pattern for anything numeric/factual.
- **Prompt rules are part of correctness**, not polish — the no-date rule changed the answers materially.
- **Independent review catches what self-review misses** — a bug hidden in a pandas default surfaced only under a dedicated edge-case lens.
- **Schema design steers the model** — `enum`s and `minimum` reduce bad tool calls at the source.

---

11. How to run (summary)

Web app: `./.venv/bin/python app.py` → http://127.0.0.1:5002. Notebook: `./.venv/bin/jupyter notebook SpendSense_Chatbot.ipynb` (kernel "Python (SpendSense)"). Full details in `README.md`. Requires an OpenAI key in `OPENAI_API_Key.txt`.

---

12. Repository structure

```
SpendSenseAI/
├── data/procurement_spend.csv   synthetic spend data (2,600 POs)
├── make_data.py                 reproducible data generator
├── spend_functions.py           6 analytics functions + tool schemas + dispatcher
├── spend_chat.py                moderation + function-calling loop
├── app.py                       Flask web app (SAP Fiori UI)
├── templates/index.html         chat UI (AJAX + Markdown renderer)
├── SpendSense_Chatbot.ipynb     teaching notebook
├── build_notebook.py            regenerates the notebook
├── ARCHITECTURE.html            architecture / flow diagram
├── README.md                    run & replication guide
└── REPORT.md / REPORT.html      this report
```

---

13. Appendix — example interaction

> **User:** How much of our spend is off-contract, and which category is worst? **SpendSense:** *[calls `maverick_spend(group_by="category")` and `get_total_spend()`]* Off-contract (maverick) spend totals **$117.5M**, about **28%** of the **$417.4M** total. The worst category is **Raw Materials** (~$32.5M of maverick spend).